

Open Image Signal Processor Profession

Project Edition

October 8, 2025

About this Document

Purpose. This document records the algorithm design, data flow, and tunable parameters of the Open-ISP Pro project. It can be used as a reference for learners and friends in the ISP field to study.

Scope. End-to-end RAW $\rightarrow$ RGB $\rightarrow$ YUV pipeline: DPC, BLC, RAW-BM3D, AWB, DEMOSAIC, CCM, Gamma, CSC, Tone Mapping, optional YUV-CBM3D, Sharpen (EE), Saturation, Contrast, Hue, and YUV2JPEG.

Non-Goals. AE/AF details, driver-level implementation, hardware register maps, training and datasets.

Conventions. Bold uppercase for matrices/vectors; lowercase for scalars; code denotes implementation names or CLI options. Unless noted, RAW is de-blackened linear, $Y'CbCr$ follows BT.709 / Full Range.

Revision History

Version	Release Date	Change Description
1.00	2025-10-05	Initial Draft –Algorithm Development Model V1

Contents

About this Document	i
Revision History	ii
1 Introduction	1
2 Motivation	2
3 Modules	3
3.1 Dead Pixel Correction (DPC)	3
3.1.1 Notation	3
3.1.2 Step 1: Robust Noise Scale	4
3.1.3 Step 2: Threshold Resolution	4
3.1.4 Step 3: Outlier Vote (Same-Color Rule)	4
3.1.5 Step 4: Edge-Respecting Repair	4
3.1.6 Step 5: Range Guard and Mask	4
3.1.7 Remarks	4
3.2 Black Level Correction (BLC)	5
3.2.1 Mathematical Formulation	5
3.2.2 Black/White Level Estimation (Robust)	6
3.2.3 Engineering Notes	6
3.3 Bayer Noise Reduction (RAW-BM3D)	7
3.3.1 Noise Model and Estimation	9

3.3.2	Block Matching and Grouping	9
3.3.3	3D Transform and Basic (Hard-Threshold) Estimate	9
3.3.4	Wiener Refinement (Final Estimate)	10
3.3.5	Aggregation with Kaiser Weights	10
3.3.6	Channel-Wise Adaptation and Engineering Notes	10
3.4	Auto White Balance (AWB)	11
3.4.1	Notation	12
3.4.2	Step 1: Gated Sampling	12
3.4.3	Step 2: Adaptive Top%	12
3.4.4	Step 3: Per-Channel Bright Means	12
3.4.5	Step 4: Gains, Warmth and Rails	13
3.4.6	Step 5: Application and Clipping	13
3.4.7	Remarks	13
3.5	Demosaic	13
3.5.1	Notation	15
3.5.2	Step 1: Green (G) at R/B Sites via Direction-Aware Interpolation	15
3.5.3	Step 2: R/B at G Sites via Color-Difference Propagation	16
3.5.4	Step 3: Cross-Color at Opposite Sites (Diagonal Interpolation)	16
3.5.5	Step 4: Hue-Preserving Ratio Guard (Optional)	17
3.5.6	Remarks (Engineering Choices)	17
3.6	Color Correction Matrix (CCM)	18
3.6.1	Notation	18
3.6.2	Step 1: Build Cam→XYZ	18
3.6.3	Step 2: Illuminant Interpolation (Mired)	18
3.6.4	Step 3: Chromatic Adaptation	19
3.6.5	Step 4: Compose and Normalize	19
3.6.6	Step 5: Fallbacks and “Cheat”	19
3.6.7	Remarks	19
3.7	Gamma Correction	19
3.7.1	Notation	20
3.7.2	Step 1: Normalize and Clamp	20
3.7.3	Step 2: Soft Negative Lift & Power	20
3.7.4	Step 3: Rescale and Bound	20
3.7.5	Step 4: Preview (QA)	20
3.7.6	Step 5: Quantization for Saving	20
3.7.7	Remarks	21
3.8	Tone Mapping	21
3.8.1	Global Exposure (Log-Average Anchor)	21

3.8.2	Filmic Curves (Hable / ACES Approx)	21
3.8.3	White-Point Normalization	23
3.8.4	Hue-Preserving Brightness Scaling (Proportional Color)	23
3.8.5	Soft Gamut Convergence (Soft Knee to Luminance)	23
3.8.6	Optional Local-Lite Contrast (Edge-Limited)	23
3.8.7	Output	24
3.9	Color Space Conversion (CSC)	24
3.9.1	Notation	24
3.9.2	Step 1: Pre-normalization to 8-bit $R'G'B'$	24
3.9.3	Step 2: $RGB \rightarrow Y'CbCr$	25
3.9.4	Step 3: Range Mapping (full $\leftrightarrow$ limited)	25
3.9.5	Step 4: Planar 4:4:4 Output	25
3.9.6	Remarks	25
3.10	Sharpening (EE)	26
3.10.1	Notation and Detail Layer	27
3.10.2	Noise-Gated Detail (MAD Threshold)	27
3.10.3	Edge Mask (Gradient-Weighted)	27
3.10.4	Tone Protection (Shadows/Highlights)	28
3.10.5	Asymmetric Halo Limiting	28
3.10.6	Final Composition	28
3.11	Saturation	29
3.11.1	Notation	29
3.11.2	Step 1: Center and Scale	29
3.11.3	Step 2: Re-encode and Clip	30
3.11.4	Remarks	30
3.12	Contrast	30
3.12.1	Step 1: Linear Around a Midpoint	31
3.12.2	Step 2: Sigmoid S-Curve	31
3.12.3	Step 3: Power-Law (Gamma) Shaping	31
3.12.4	Step 4: Histogram Strategies	31
3.12.5	Remarks	31
3.13	Hue	31
3.13.1	Notation	32
3.13.2	Step 1: $YUV \leftrightarrow RGB$ Consistency	32
3.13.3	Step 2: Global Hue Rotation	32
3.13.4	Step 3: Selective Rotation (Windowed Blend)	32
3.13.5	Step 4: Warm Highlights (Optional)	32
3.13.6	Remarks	32

3.14 YUV2JPEG	33
3.14.1 Color Standard & Range Normalization	34
3.14.2 Optional 444→420 Downsampling	34
3.14.3 MCU Layout and Padding	34
3.14.4 Level Shift and 2D AAN FDCT	35
3.14.5 Quantization and Quality Control	35
3.14.6 ZigZag Reorder, DC Differencing and AC RLE	35
3.14.7 Huffman Entropy Coding	36
3.14.8 Bitstream Packing and Byte Stuffing	36
3.14.9 JFIF Segments and Sampling Factors	36
References	37

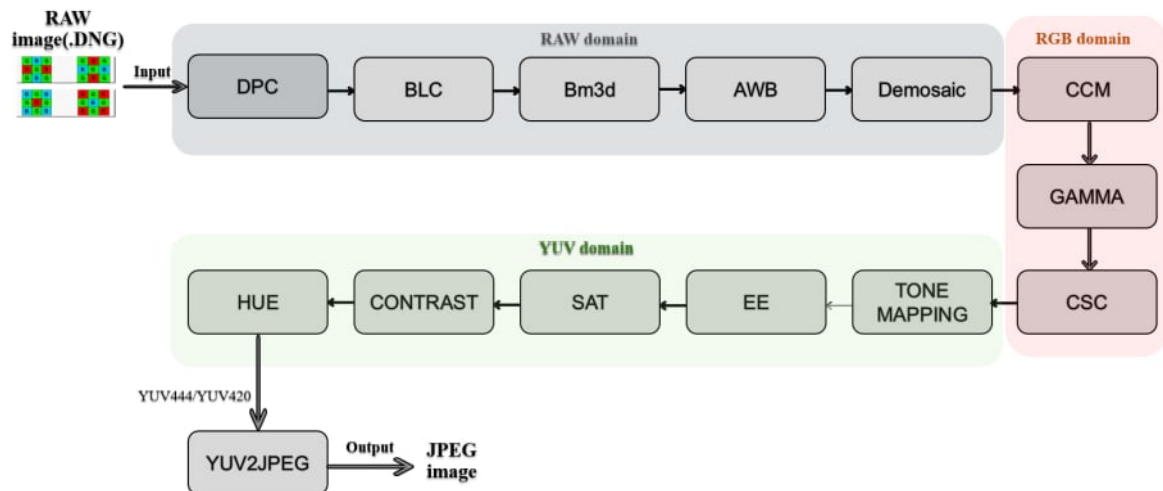
List of Tables

3.1	Parameter quick reference —BLC	7
3.2	Parameter quick reference —Tone Mapping	24
3.3	CSC parameters and typical settings	25
3.4	Parameter quick reference —EE	29
3.5	Parameter quick reference —YUV2JPEG	36

Chapter 1

Introduction

OPENISP_PRO is a **pure-Python** software ISP that converts **RAW/DNG** from image sensors into display-ready **RGB/YUV** and **JPEG** outputs. The pipeline follows a practical, production-minded order—**RAW input** → **front-end calibration** → **color reconstruction** → **back-end enhancement** → **encoding**—with robust fallbacks, metadata propagation, and intermediate artifacts for quality control and reproducibility.



Chapter 2

Motivation

We are just stepping into ISP and computational photography. Our skills are still being honed, but we're committed to doing the work right. This open-source project has a modest aim: to organize common workflows, make careful, incremental improvements to existing methods, and provide clear, reusable implementations. We believe solid engineering practice and reproducible experiments make algorithms more robust and easier to use. The project follows an open, collaborative model—feedback and contributions are welcome—and we will iterate continuously, from detail refinements to algorithm exploration, turning “it works” into “it works better”, step by step.

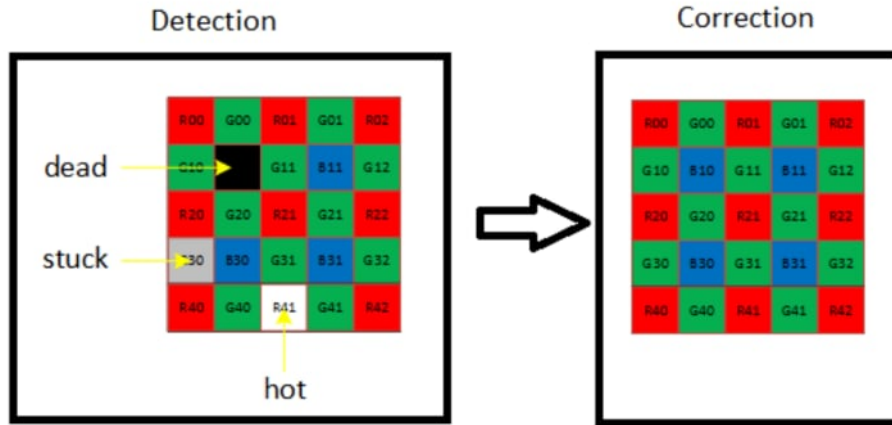
Chapter 3

Modules

3.1 Dead Pixel Correction (DPC)

Overview

We detect and repair defective RAW samples on the Bayer mosaic by comparing each site to its *same-color* neighbors. A robust noise scale anchors the decision threshold, and repairs favor edge-consistent interpolation to avoid texture blur. The stage outputs both a corrected RAW and a binary mask for traceability.



3.1.1 Notation

Let $x \in Z^{H \times W}$ be the Bayer RAW and WL the sensor white level. Each pixel $p_0 = x_{i,j}$ belongs to $c \in \{R, G, B\}$ by a 2×2 CFA LUT. Define the same-color 8-neighborhood $\mathcal{N}_{\text{same}}^{(8)}(i, j)$ by stride-2 offsets aligned to c . For neighbors $\{q_u\}_{u=1}^8$, we write absolute differences $d_u = |q_u - p_0|$.

3.1.2 Step 1: Robust Noise Scale

We infer per-sample noise σ from opposite-pair differences to suppress low-frequency structure:

$$\mathcal{D} = \{|q_2 - q_7|, |q_4 - q_5|, |q_1 - q_8|, |q_3 - q_6|\}, \quad \sigma \approx \frac{\text{MAD}(\mathcal{D})}{0.6745\sqrt{2}}.$$

3.1.3 Step 2: Threshold Resolution

We unify three modes by a single threshold T :

$$T = \begin{cases} \text{ABS: } \tau & (\text{LSB}) \\ \text{REL: } \tau \cdot \text{WL} & \text{then } T \leftarrow \text{clip}(T, 0.001 \text{ WL}, 0.05 \text{ WL}). \\ \text{AUTO: } k_{\text{auto}} \sigma, k_{\text{auto}} \in [5, 8] \end{cases}$$

3.1.4 Step 3: Outlier Vote (Same-Color Rule)

A site is defective if at least M neighbors exceed the threshold:

$$C = \sum_{u=1}^8 1[d_u > T], \quad C \geq M, \quad M \in \{1, \dots, 8\}.$$

3.1.5 Step 4: Edge-Respecting Repair

We reconstruct $\tilde{p}_0$ by one of three consistent estimators:

Gradient: $\tilde{p}_0$ = average along the minimum-variance axis, **Mean:** $\tilde{p}_0$ = same-color cross m

The choice balances edge fidelity (gradient) and texture robustness (median).

3.1.6 Step 5: Range Guard and Mask

We clip to sensor range and record the decision:

$$\hat{p}_0 = \text{clip}(\tilde{p}_0, 0, \text{WL}), \quad m_{i,j} = 1[C \geq M].$$

3.1.7 Remarks

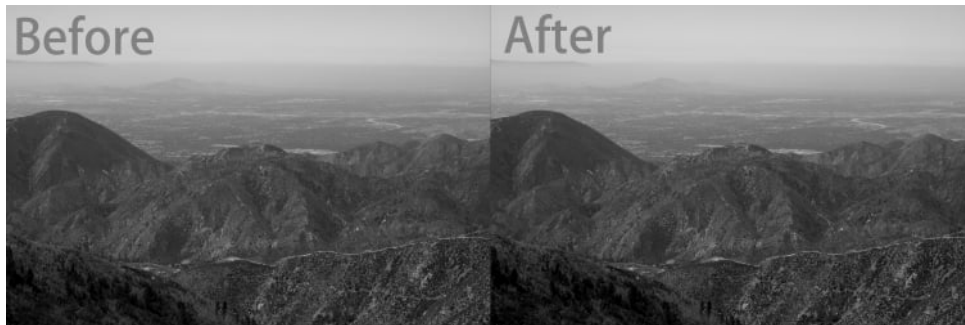
- (i) Clamping T prevents unstable behavior in ultra-clean or very noisy scenes. (ii) $M=8$ is conservative; $M=6$ captures small clusters. (iii) For green sites we keep

unit-diagonal pairs to preserve same-color parity. (iv) The preview (percentile + gamma) is QA-only and never influences decisions.

3.2 Black Level Correction (BLC)

Overview

Remove the sensor's zero-illumination offset on each CFA sub-grid (R/Gr/Gb/B), then safely normalize to $[0, 1]$ per channel. Hot pixels and defect rows/columns are excluded when estimating black levels to avoid over-subtraction.



3.2.1 Mathematical Formulation

Let $x_{i,c}$ be the RAW value at pixel i and color $c \in \{R, Gr, Gb, B\}$. The per-channel black level is b_c , and the per-channel white level is w_c (from metadata if available; otherwise $w_c = 2^n - 1$ for n -bit data). The effective dynamic range is $w_c - b_c$.

$$\tilde{x}_{i,c} = \max(0, x_{i,c} - b_c) \quad (3.1)$$

To avoid division instability, define a safe denominator

$$d_c = \max(w_c - b_c, \varepsilon), \quad \varepsilon \approx 10^{-6}. \quad (3.2)$$

Per-channel normalization to $[0, 1]$:

$$y_{i,c} = \text{clip}\left(\frac{\tilde{x}_{i,c}}{d_c}, 0, 1\right). \quad (3.3)$$

3.2.2 Black/White Level Estimation (Robust)

Seeds and grid search. For each c , form candidate black levels from (i) the darkest- $p\%$ pixels (e.g., $p \in [0.8, 2]$) as a seed $\tilde{b}_c^{(p)}$, (ii) optional metadata default, then apply small scales $s \in \{0.95, 1.00, 1.05\}$:

$$\mathcal{B}_c = \{s \cdot \tilde{b}_c^{(k)} \mid s \in \{0.95, 1.00, 1.05\}, \tilde{b}_c^{(k)} \in \{\text{percentile seed, metadata}\}\}. \quad (3.4)$$

Exclusion mask. Define a mask Ω_c (ROI in dark regions) that excludes hot pixels / defects so that outliers do not bias the estimate.

Over-subtraction penalty. For a candidate b_c , measure the negative-rate within ROI after (3.1):

$$p_{\text{neg}}(b_c) = \frac{|\{i \in \Omega_c : x_{i,c} - b_c < 0\}|}{|\Omega_c|}. \quad (3.5)$$

Objective and selection. Rank candidates by a penalized score (smaller is better), favoring low p_{neg} and uniform dark-field residuals:

$$J_c(b_c) = \lambda_{\text{neg}} p_{\text{neg}}(b_c) + \lambda_{\text{res}} r_c(b_c), \quad b_c^* = \arg \min_{b_c \in \mathcal{B}_c} J_c(b_c), \quad (3.6)$$

where $r_c(b_c)$ summarizes low-frequency residual variation in dark ROIs (smaller is better). Enforce the safety constraint

$$0 \leq b_c^* < w_c. \quad (3.7)$$

Select top- K candidates by (3.6) for a full-frame recheck, then finalize b_c^* .

3.2.3 Engineering Notes

- **Channel-wise processing:** split RAW into CFA sub-grids (R/Gr/Gb/B) and estimate $\{b_c\}$ independently to avoid color bias.
- **ROI sampling:** evaluate on dark ROIs for speed, then verify the top- K on the full frame to ensure robustness.

- **Safety:** clamp per (3.1) and (3.2); $\log p$, scale s , final b_c^* , and p_{neg} for traceability.
- **Throughput:** per-channel constants (b_c^*, d_c) enable vectorized normalization via (3.3).

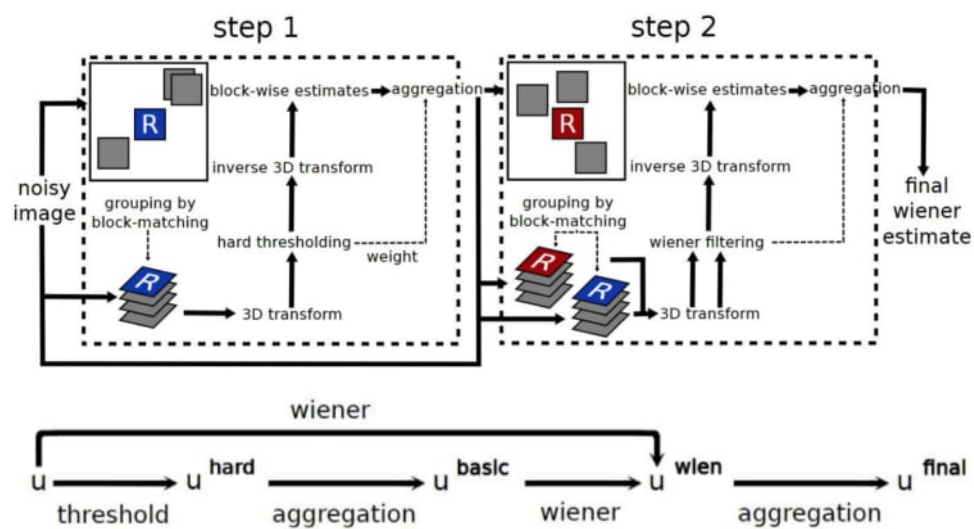
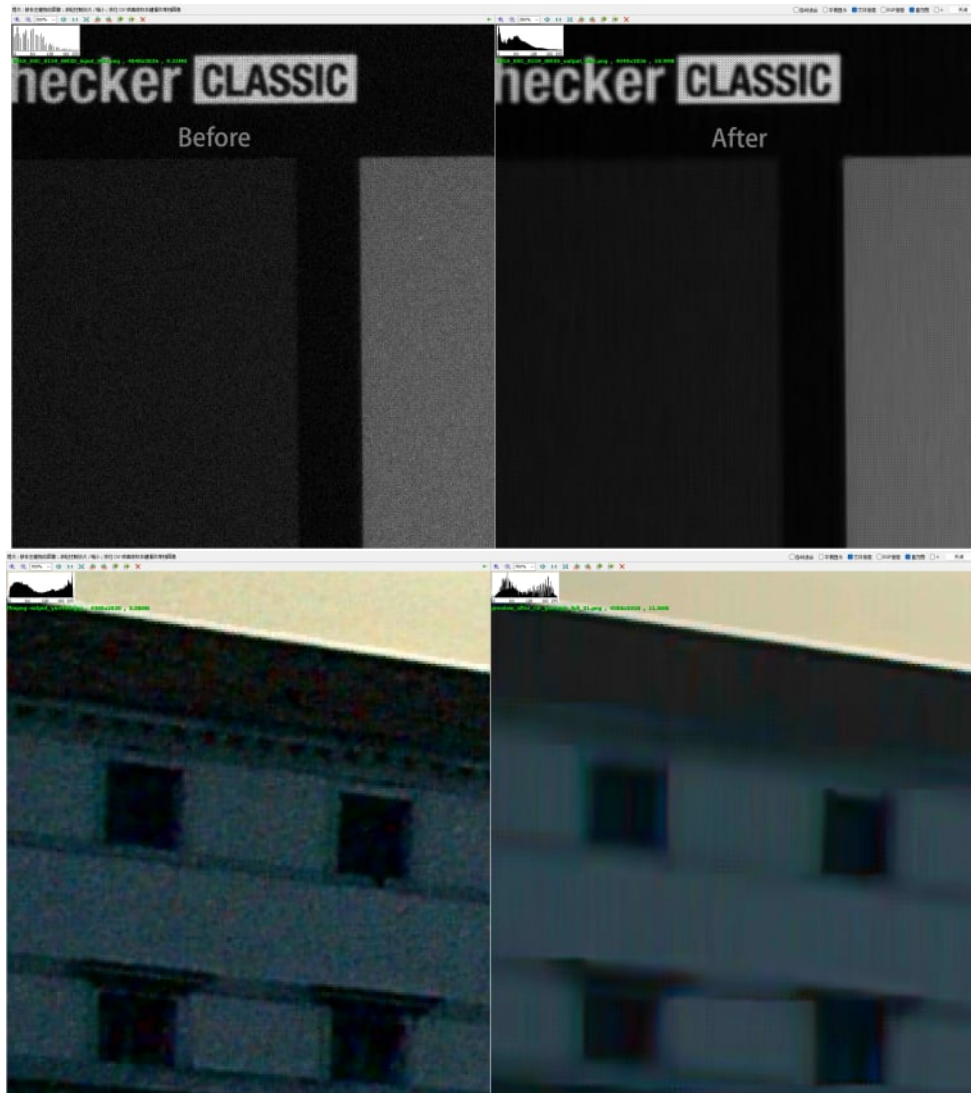
Table 3.1 Parameter quick reference —BLC

Parameter	Purpose	Recommended Range
p_percent	Darkest-percentile seed	0.8 % to 2 %
scales	Seed scaling set s	{0.95, 1.00, 1.05}
use_meta_BL	Use metadata black level seed	true/false
neg_weight	λ_{neg} in (3.6)	higher $\Rightarrow$ safer
roi_size	Dark ROI fraction	small (fast)
topK	Candidates for full-frame recheck	3–5

3.3 Bayer Noise Reduction (RAW-BM3D)

Overview

We denoise each Bayer sub-grid ($c \in \{R, Gr, Gb, B\}$) *before* demosaicing using a two-stage BM3D: (i) Basic (hard-thresholding in a collaborative 3D transform domain) and (ii) Wiener refinement guided by the Basic estimate. Blocks are matched non-locally, stacked, transformed (2D DCT in-block + 1D Haar across blocks), then inverse-transformed and aggregated with Kaiser window weights to suppress block boundaries. Channel-wise noise σ_c is robustly estimated and used to set thresholds and Wiener gains.



3.3.1 Noise Model and Estimation

For channel c , the RAW observation is

$$x_{i,c} = s_{i,c} + n_{i,c}, \quad n_{i,c} \sim \mathcal{N}(0, \sigma_c^2). \quad (3.8)$$

Estimate σ_c via MAD on a high-pass (e.g., finite difference) response:

$$\sigma_c = \frac{\text{MAD}((D * x_c)_i)}{0.6745}, \quad \text{MAD}(z) = \text{median}(|z - \text{median}(z)|). \quad (3.9)$$

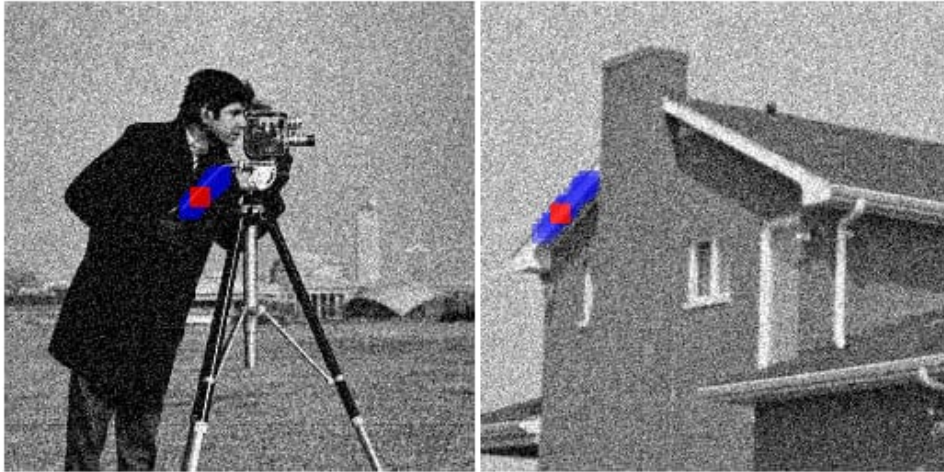
Optionally normalize to unit variance for stable matching/thresholding: $z_{i,c} = x_{i,c} / \sigma_c$.

3.3.2 Block Matching and Grouping

For each reference location i , extract a vectorized patch $P_{i,c} \in R^m$ ($m = b \times b$) and search a window $\mathcal{W}_i$ for similar patches. Select top- K indices

$$\mathcal{S}_{i,c} = \arg \min_{\substack{S \subset \mathcal{W}_i \\ |S|=K}} \sum_{j \in S} \|P_{i,c} - P_{j,c}\|_2^2, \quad \text{or} \quad \mathcal{S}_{i,c} = \{j \in \mathcal{W}_i : \|P_{i,c} - P_{j,c}\|_2^2 \leq \tau_c\}. \quad (3.10)$$

Stack them into a 3D group tensor $G_{i,c} \in R^{b \times b \times K}$.



3.3.3 3D Transform and Basic (Hard-Threshold) Estimate

Apply a separable 3D transform: 2D orthonormal transform T_{2D} (e.g., DCT) within each patch and 1D Haar T_{1D} along the grouping dimension:

$$C_{i,c} = (T_{1D} \circ T_{2D})(G_{i,c}). \quad (3.11)$$

Hard-threshold the coefficients using $\lambda_{\text{hard}}\sigma_c$:

$$\hat{C}_{i,c}^{(B)} = C_{i,c} \cdot \mathbf{1}(|C_{i,c}| \geq \lambda_{\text{hard}}\sigma_c), \quad (3.12)$$

then invert the transforms to obtain the grouped Basic estimate $\hat{G}_{i,c}^{(B)}$. A Kaiser window $W \in R^{b \times b}$ is applied on each patch to reduce seams.

3.3.4 Wiener Refinement (Final Estimate)

Repeat the grouping on the Basic image to form $G_{i,c}$ and its *reference* transform $C_{i,c}^{\text{ref}}$. Compute Wiener gains

$$H_{i,c} = \frac{|C_{i,c}^{\text{ref}}|^2}{|C_{i,c}^{\text{ref}}|^2 + \sigma_c^2}, \quad \tilde{C}_{i,c}^{(W)} = H_{i,c} \odot C_{i,c}, \quad (3.13)$$

and invert to get $\hat{G}_{i,c}^{(W)}$. Here $\odot$ is element-wise multiplication.

3.3.5 Aggregation with Kaiser Weights

Each reconstructed group contributes overlapping patches back to the image. Let $\mathcal{N}(p)$ be all groups/patches covering pixel p . The aggregated estimate for channel c is

$$\hat{x}_{p,c} = \frac{\sum_{(i,k) \in \mathcal{N}(p)} \omega_{i,c} W_{p-i} \hat{P}_{i,k,c}^{(\cdot)}}{\sum_{(i,k) \in \mathcal{N}(p)} \omega_{i,c} W_{p-i}}, \quad (3.14)$$

where $\hat{P}_{i,k,c}^{(\cdot)}$ is the (Basic or Wiener) reconstructed patch sample at p , W is the Kaiser window, and the scalar group weight is typically

$$\omega_{i,c}^{(B)} = \frac{1}{\epsilon + \#\{|C_{i,c}| \geq \lambda_{\text{hard}}\sigma_c\}}, \quad \omega_{i,c}^{(W)} = \frac{1}{\epsilon + \sum H_{i,c}^2}. \quad (3.15)$$

The final RAW estimate per channel is taken from the Wiener stage.

3.3.6 Channel-Wise Adaptation and Engineering Notes

- **Per-CFA processing:** process $R/Gr/Gb/B$ independently with their own σ_c and thresholds; optional pre-whitening $x_c \mapsto x_c/\sigma_c$ stabilizes matching.
- **Speed/quality:** reduce search window or K for speed; increase block step for further speedups at slight quality loss.

- **Artifacts:** prior DPC/BLC removes outliers/offsets that would otherwise bias matching; Kaiser aggregation suppresses blockiness.
- **Parameters (typical):** $b \in [8, 12]$, search window $\in [16, 32]$, $\lambda_{\text{hard}} \in [2.5, 3.5]$, $K \in [8, 16]$.

3.4 Auto White Balance (AWB)

Overview

We estimate the illuminant directly in the RAW/Bayer domain after BLC and denoising. Grounded in the *Gray-World* assumption—that after correct white balance the spatial average reflectance of a typical scene is achromatic—we treat green as the neutral anchor. The method samples a controlled Top% of bright pixels per color plane in a de-blackened “stats domain”, derives gains relative to green, applies a gentle warm bias, and keeps chroma excursions within guardrails. All decisions are logged for reproducibility; previews remain diagnostic only.

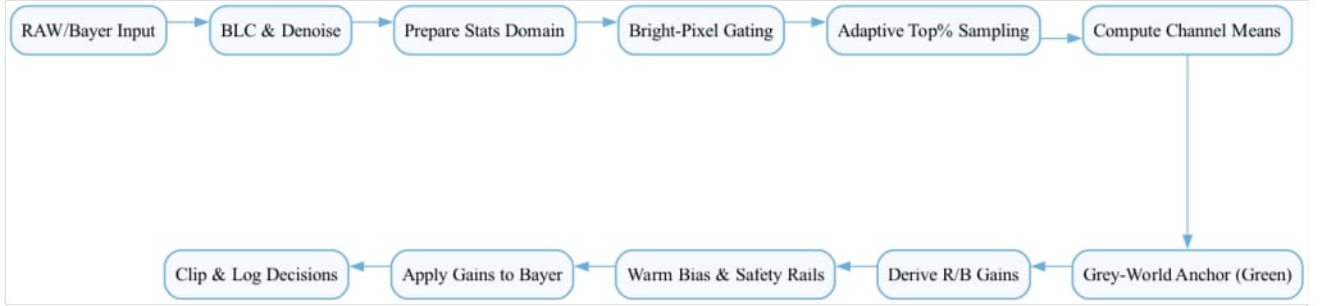


3.4.1 Notation

Let (R, G_1, G_2, B) be Bayer planes and b the BLC offset. We define the stats-domain ReLU signals

$$R_f = \max(R-b, 0), \quad G_{1f} = \max(G_1-b, 0), \quad G_{2f} = \max(G_2-b, 0), \quad B_f = \max(B-b, 0),$$

and stabilized green $G_f = \frac{1}{2}(G_{1f} + G_{2f})$; white level is WL.



3.4.2 Step 1: Gated Sampling

Low/high gates in WL fraction, $(\lambda_{\text{low}}, \lambda_{\text{high}})$, map to

$$\ell = \max(\lambda_{\text{low}}\text{WL} - b, 0), \quad u = \max(\lambda_{\text{high}}\text{WL} - b, 0),$$

and we discard samples outside (ℓ, u) .

3.4.3 Step 2: Adaptive Top%

We choose a baseline Top% from exposure tiers and adapt it by saturation incidence and bright-end spread; the result p is clamped into $[p_{\text{min}}, p_{\text{max}}]$ to avoid corner cases.

3.4.4 Step 3: Per-Channel Bright Means

For $X \in \{R_f, G_{1f}, G_{2f}, B_f\}$, we compute μ_X as the mean of the top $p\%$ values within (ℓ, u) . Define

$$E_R = \mu_{R_f}, \quad E_G = \frac{1}{2}(\mu_{G_{1f}} + \mu_{G_{2f}}), \quad E_B = \mu_{B_f}.$$

3.4.5 Step 4: Gains, Warmth and Rails

With green as anchor,

$$g_R^* = \frac{E_G}{E_R}, \quad g_B^* = \frac{E_G}{E_B}, \quad g_G = 1.$$

A mild warm bias ω increases g_R and decreases g_B slightly; we constrain $g_B \leq \rho_{\max} g_R$ and clamp both into $[0.5, 3.0]$.

3.4.6 Step 5: Application and Clipping

We apply gains to the Bayer planes in place and clip to $[0, \text{WL}]$. We also persist p , gains, gates and basic statistics for downstream audit.

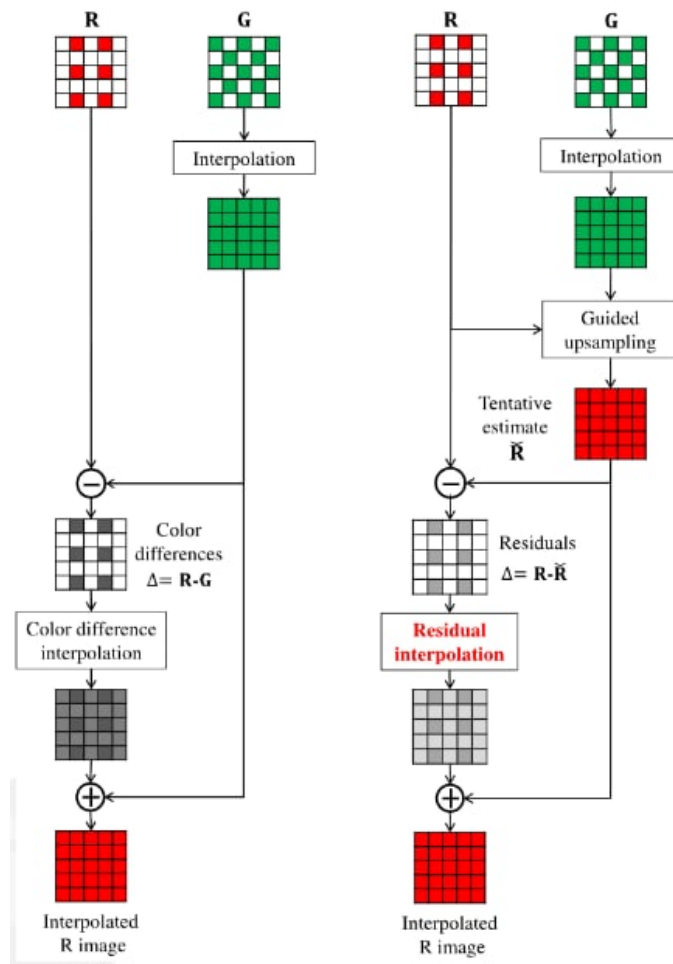
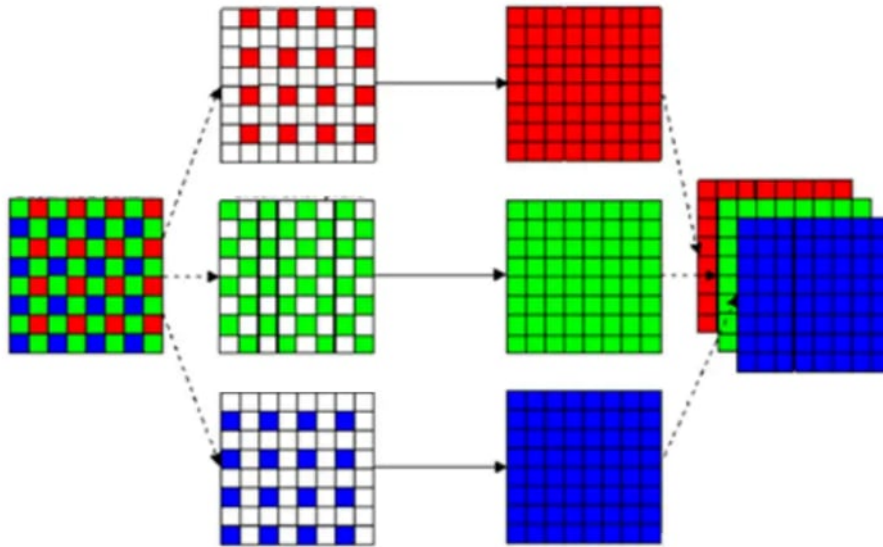
3.4.7 Remarks

(i) The ReLU stats domain eliminates residual negative bias from BLC rounding. (ii) Adaptive Top% balances specular contamination against noise variance. (iii) Green averaging stabilizes noisy per-quadrant estimates while keeping exposure unchanged. (iv) The warm bias gently counteracts overly cool low-CCT scenes without forcing a look.

3.5 Demosaic

Overview

We reconstruct full-resolution RGB from a Bayer CFA by first completing the green channel (luminance proxy) with direction-aware interpolation, then restoring red/blue via color-difference constancy. This reduces zippering and false color on edges while preserving fine textures. The method blends ideas from gradient/homogeneity-directed demosaicing (AHD) and adaptive residual interpolation (ARI).



3.5.1 Notation

Let R, G, B denote the latent full-resolution channels and $\mathcal{M}_R, \mathcal{M}_{G_r}, \mathcal{M}_{G_b}, \mathcal{M}_B$ be the CFA masks for an RRGB pattern. The observed mosaic is

$$\mathbf{m} = \mathcal{M}_R R + \mathcal{M}_{G_r} G + \mathcal{M}_{G_b} G + \mathcal{M}_B B. \quad (3.16)$$

We write pixel location i and its horizontal/vertical/diagonal neighbors by $i-x, i+x, i-y, i+y, i-d, i+d$ respectively.

3.5.2 Step 1: Green (G) at R/B Sites via Direction-Aware Interpolation

At an R site (B analogous), form two candidates using color-difference constancy ($R - G$ varies slowly) along horizontal and vertical directions:

$$\hat{G}_i^{(H)} = \frac{G_{i-x} + G_{i+x}}{2} + \frac{2R_i - R_{i-x} - R_{i+x}}{4}, \quad (3.17)$$

$$\hat{G}_i^{(V)} = \frac{G_{i-y} + G_{i+y}}{2} + \frac{2R_i - R_{i-y} - R_{i+y}}{4}. \quad (3.18)$$

Compute adaptive directional costs (AHD-style) with chroma-compensated gradients:

$$\Delta_H = |G_{i-x} - G_{i+x}| - |2R_i - R_{i-x} - R_{i+x}|, \quad (3.19)$$

$$\Delta_V = |G_{i-y} - G_{i+y}| - |2R_i - R_{i-y} - R_{i+y}|. \quad (3.20)$$

Convert to soft weights and fuse to avoid hard switches:

$$w_H = \frac{1}{(|\Delta_H| + \kappa)^\alpha}, \quad w_V = \frac{1}{(|\Delta_V| + \kappa)^\alpha}, \quad \hat{G}_i = \frac{w_H \hat{G}_i^{(H)} + w_V \hat{G}_i^{(V)}}{w_H + w_V}, \quad (3.21)$$

with small $\kappa > 0$ and $\alpha \in [1, 2]$. In near-flat regions ($|\Delta_H - \Delta_V| < \tau$), the fusion approximates a simple average for stability.

3.5.3 Step 2: R/B at G Sites via Color-Difference Propagation



At a G site, propagate color differences along the less-variant direction. For red:

$$\hat{R}_i^{(H)} = G_i + \frac{(R_{i-x} - G_{i-x}) + (R_{i+x} - G_{i+x})}{2}, \quad (3.22)$$

$$\hat{R}_i^{(V)} = G_i + \frac{(R_{i-y} - G_{i-y}) + (R_{i+y} - G_{i+y})}{2}. \quad (3.23)$$

Choose by the same directional weights as in (3.21) (recomputed on G), and set

$$\hat{R}_i = \frac{w_H \hat{R}_i^{(H)} + w_V \hat{R}_i^{(V)}}{w_H + w_V}, \quad \hat{B}_i = \frac{w_H \hat{B}_i^{(H)} + w_V \hat{B}_i^{(V)}}{w_H + w_V}. \quad (3.24)$$

3.5.4 Step 3: Cross-Color at Opposite Sites (Diagonal Interpolation)

At a B site (symmetric for R), estimate R along the two diagonals:

$$\hat{R}_i^{(D_1)} = \frac{R_{i-d} + R_{i+d}}{2} + \frac{2G_i - G_{i-d} - G_{i+d}}{4}, \quad (3.25)$$

$$\hat{R}_i^{(D_2)} = \frac{R_{i-\bar{d}} + R_{i+\bar{d}}}{2} + \frac{2G_i - G_{i-\bar{d}} - G_{i+\bar{d}}}{4}, \quad (3.26)$$

with diagonal costs

$$\Delta_{D_1} = |R_{i-d} - R_{i+d}| - |2G_i - G_{i-d} - G_{i+d}|, \quad \Delta_{D_2} = |R_{i-\bar{d}} - R_{i+\bar{d}}| - |2G_i - G_{i-\bar{d}} - G_{i+\bar{d}}|. \quad (3.27)$$

Fuse by $w_{D_k} \propto 1/(|\Delta_{D_k}| + \kappa)^\alpha$ and set

$$\hat{R}_i = \frac{w_{D_1} \hat{R}_i^{(D_1)} + w_{D_2} \hat{R}_i^{(D_2)}}{w_{D_1} + w_{D_2}}, \quad \hat{B}_i \text{ at } R \text{ site by symmetry.} \quad (3.28)$$

3.5.5 Step 4: Hue-Preserving Ratio Guard (Optional)

To prevent chroma overshoot near high-contrast edges, constrain color ratios within $[\rho_{\min}, \rho_{\max}]$:

$$\rho_R = \frac{\hat{R}_i}{\hat{G}_i}, \quad \rho_B = \frac{\hat{B}_i}{\hat{G}_i}, \quad \tilde{\rho}_R = \text{clip}(\rho_R, \rho_{\min}, \rho_{\max}), \quad \tilde{\rho}_B = \text{clip}(\rho_B, \rho_{\min}, \rho_{\max}), \quad (3.29)$$

and update $R_i \leftarrow \tilde{\rho}_R \hat{G}_i$, $B_i \leftarrow \tilde{\rho}_B \hat{G}_i$. This keeps hue while shrinking excessive chroma length. Typical bounds: $\rho_{\min} \approx 1/4$, $\rho_{\max} \approx 4$.

3.5.6 Remarks (Engineering Choices)

(i) Use reflect padding; (ii) compute direction metrics on a G -proxy image for stability; (iii) optionally precompute a fast linear seed (e.g., MHC 5×5 kernels) and fuse it back in flat regions; (iv) in high noise, increase κ and lower α to weaken directionality.



3.6 Color Correction Matrix (CCM)

Overview

We map camera-linear RGB to a target working space (e.g., linear sRGB/AdobeRGB) with priority on camera metadata, careful illuminant interpolation, chromatic adaptation, and neutral-axis preservation. When metadata are absent, conservative fallbacks blend toward identity to minimize color-cast risk.



3.6.1 Notation

Color vectors are columns $c_{\text{cam}}, c_{\text{xyz}}, c_{\text{rgb}} \in R^3$. Matrices $M \in R^{3 \times 3}$ compose as post-multiplications on column vectors. Target space provides $M_{\text{rgb} \rightarrow \text{xyz}}$ and $M_{\text{xyz} \rightarrow \text{rgb}}$ and reference white W_t .

3.6.2 Step 1: Build Cam→XYZ

We prefer ForwardMatrix (+ optional CameraCalibration) to obtain Cam→XYZ; if unavailable, we invert the ColorMatrix path. We guard inversions by condition number and fall back to pseudo-inverse when needed.

3.6.3 Step 2: Illuminant Interpolation (Mired)

Given two calibration illuminants at T_1, T_2 and a shot CCT T_s , we interpolate in mired space:

$$m(T) = 10^6/T, \quad \alpha = \text{clip}\left(\frac{m(T_s) - m(T_1)}{m(T_2) - m(T_1)}, 0, 1\right),$$

$$M_{\text{cam} \rightarrow \text{xyz}} = (1 - \alpha)M_{\text{cam} \rightarrow \text{xyz}}^{(1)} + \alpha M_{\text{cam} \rightarrow \text{xyz}}^{(2)}.$$

3.6.4 Step 3: Chromatic Adaptation

Assuming D50-referenced camera matrices, we adapt XYZ to the target white via Bradford:

$$M_{\text{CAT}}(D50 \rightarrow W_t) = B^{-1} \text{diag}\left(\frac{BW_t}{BW_{D50}}\right) B.$$

3.6.5 Step 4: Compose and Normalize

We compose

$$M_{\text{cam} \rightarrow \text{rgb}} = M_{\text{xyz} \rightarrow \text{rgb}} M_{\text{CAT}} M_{\text{cam} \rightarrow \text{xyz}},$$

then row-normalize to preserve the neutral axis (row sums ≈ 1) before applying to images.

3.6.6 Step 5: Fallbacks and “Cheat”

If camera matrices are missing, we form an assumed mapping (bypass/XYZ@D65/XYZ@D50), blend toward identity with a small factor, and row-normalize. Optionally we shrink any nominal matrix slightly toward identity to soften the look while staying faithful.

3.6.7 Remarks

(i) Row normalization is crucial to keep grays neutral after AWB. (ii) Mired-space interpolation matches human perception across CCT. (iii) We propagate condition numbers and CAT choice for diagnostics.

3.7 Gamma Correction

Overview

We remap linear RGB (counts) to a perceptually compressed encoding through normalization, a power-law shaping, and rescaling. Tiny negatives are lifted softly to avoid hue bias near black, and optional previews visualize the *input* through an OETF for QA.



3.7.1 Notation

Let $x \in [0, \text{WL}_{\text{in}}]$ be linear per-channel counts, $\gamma > 0$ the shaping exponent, and WL_{out} the output white. We denote normalized values by $x_n = x/\text{WL}_{\text{in}}$ and the shaped output by y .

3.7.2 Step 1: Normalize and Clamp

We normalize to $[0, 1]$ and optionally clamp to remove stray values.

3.7.3 Step 2: Soft Negative Lift & Power

We lift tiny negatives to a small $\epsilon \geq 0$ and then apply $y_n = x_n^\gamma$ for $x_n > 0$, else 0.

3.7.4 Step 3: Rescale and Bound

We rescale $y = \text{WL}_{\text{out}} y_n$ and clamp to $[0, \text{WL}_{\text{out}}]$ if desired.

3.7.5 Step 4: Preview (QA)

For visualization, we show the *input* x_n under the sRGB OETF; this does not affect the numeric output.

3.7.6 Step 5: Quantization for Saving

When saving to 8–16 bit files, we quantize the normalized output to the target bit depth while keeping the semantic meaning of the white level unchanged unless specified.

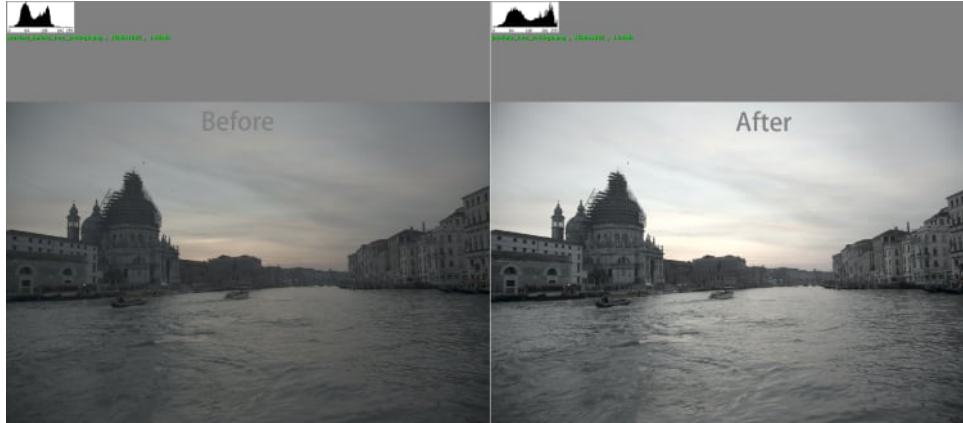
3.7.7 Remarks

(i) Choosing $\gamma < 1$ brightens mid-tones; $\gamma > 1$ compresses highlights further. (ii) Working in float with a soft floor avoids chroma skew around black. (iii) Keeping $WL_{\text{out}} = WL_{\text{in}}$ preserves consistent metadata.

3.8 Tone Mapping

Overview

We compress HDR scene-referred luminance to a display-referred range with a global tone curve (Filmic: Hable or ACES-approx), followed by white-point normalization and hue-preserving color scaling. An optional lightweight local module enhances mid-scale contrast with edge-aware limits.



3.8.1 Global Exposure (Log-Average Anchor)

Let L_i be the scene-referred linear luminance per pixel. Compute the log-average luminance and a safe exposure gain:

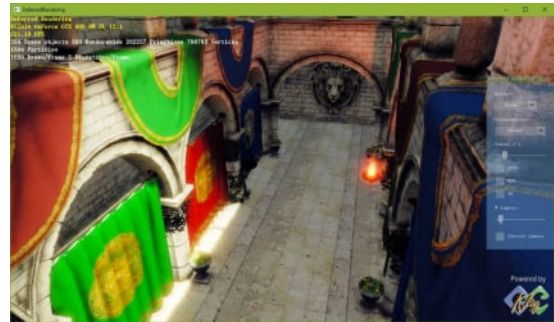
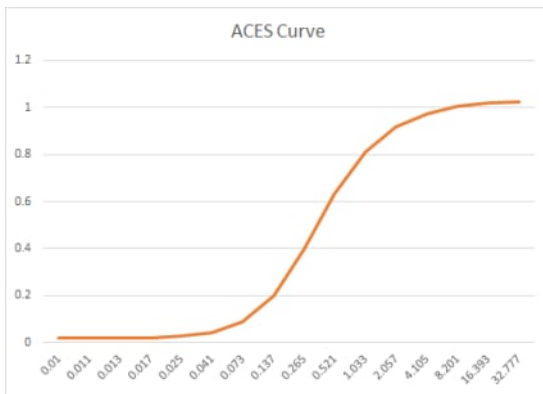
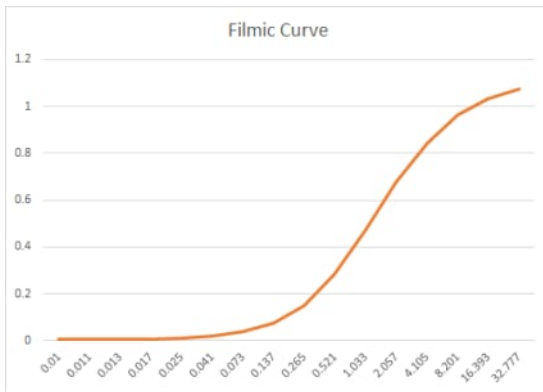
$$\bar{L}_{\log} = \exp\left(\frac{1}{N} \sum_{i=1}^N \log(\epsilon + L_i)\right), \quad k_{\text{exp}} = \text{clip}\left(\frac{L_{\text{mid}}}{\bar{L}_{\log}}, k_{\min}, k_{\max}\right), \quad (3.30)$$

$$L_i^{(0)} = k_{\text{exp}} L_i. \quad (3.31)$$

3.8.2 Filmic Curves (Hable / ACES Approx)

We map $x = L^{(0)}$ to $y = f(x)$ with a rational "filmic" S-curve:
graphicx

算法	计算复杂度	动态范围适应性	细节保留	适用场景
Hable Filmic	中	高	优 (高光)	影视、高质量游戏、摄影
Reinhard	低	中	良 (全局)	实时渲染、移动端应用
ACES Filmic	高	极高	优 (全范围)	电影工业、跨平台制作
Linear	极低	低	差	硬件资源受限场景



Hable (Uncharted2 form).

$$f_{\text{Hable}}(x) = \frac{x(Ax + CB) + DE}{x(Ax + B) + DF} - \frac{E}{F}, \quad (3.32)$$

where $A \dots F$ control toe/shoulder softness and mid-tone contrast.

ACES (approximate).

$$f_{\text{ACES}}(x) = \frac{x(ax + b)}{x(cx + d) + e}, \quad (3.33)$$

with $a \dots e$ fitted for a compact, balanced S-curve.

3.8.3 White-Point Normalization

Let q_p be the p -th luminance percentile of $\{L_i^{(0)}\}$ (e.g., $p=95$). Normalize the curve output to reach the display headroom:

$$\alpha = \frac{1}{f(q_p) + \epsilon}, \quad L_i^{(1)} = \alpha f(L_i^{(0)}), \quad (3.34)$$

where α gently restores global contrast after filmic compression.

3.8.4 Hue-Preserving Brightness Scaling (Proportional Color)

Let $\mathbf{c}_i = (R_i, G_i, B_i)$ be the linear RGB aligned with L_i and $\mathbf{c}_i^{(1)}$ the RGB consistent with $L_i^{(1)}$. Define a luminance-consistent gain and apply it to each channel to preserve hue:

$$s_i = \text{clip}\left(\frac{L_i^{(1)} + \delta}{L_i + \delta}, s_{\min}, s_{\max}\right), \quad \tilde{\mathbf{c}}_i = \mathbf{c}_i \odot s_i^{\gamma_c}, \quad (3.35)$$

optionally weakening s_i in deep shadows via a smoothstep gate $g_i \in [0, 1]$ and using $s_i \leftarrow 1 + g_i(s_i - 1)$.

3.8.5 Soft Gamut Convergence (Soft Knee to Luminance)

To avoid hard clipping on saturated highlights while keeping hue, compress channel excursions toward the local luminance axis:

$$\eta_i = \min\left(1, \frac{1 - L_i^{(1)}}{\max_c(\tilde{c}_{i,c} - L_i^{(1)})_+ + \epsilon}\right), \quad \mathbf{c}_i^{(2)} = L_i^{(1)} \mathbf{1} + \eta_i(\tilde{\mathbf{c}}_i - L_i^{(1)} \mathbf{1}). \quad (3.36)$$

3.8.6 Optional Local-Lite Contrast (Edge-Limited)

A lightweight local boost on the luminance channel avoids halos and noise amplification:

$$B_i = (\mathcal{B}_r * L^{(1)})_i, \quad d_i = \text{clip}\left(\left(\frac{L_i^{(1)}}{B_i + \epsilon}\right)^{\gamma_d}, 1 - \Delta, 1 + \Delta\right), \quad e_i = \frac{1}{1 + \kappa \|\nabla L_i^{(1)}\|^2}, \quad L_i^{(2)} = L_i^{(1)} \quad (3.37)$$

Use high-ISO gating to disable this step in noisy images.

3.8.7 Output

For direct display, apply the target OETF (e.g., sRGB) and clamp to $[0, 1]$; for further HDR/secondary grading, keep linear and only remove negatives.

Table 3.2 Parameter quick reference —Tone Mapping

Parameter	Purpose	Typical/Range
L_{mid}	Middle-gray anchor in (3.30)	0.18 (scene-referred)
$k_{\text{min}}, k_{\text{max}}$	Exposure bounds	scene/ISO dependent
Curve	Filmic type	Hable / ACES-approx
$A..F$ or $a..e$	Filmic shape (toe/shoulder)	dataset-fitted
p	White-point percentile in (3.34)	90–98
$s_{\text{min}}, s_{\text{max}}$	Proportional color bounds	0.5–2.0
γ_c	Hue-preserving strength	0.6–1.0
r, γ_d, Δ	Local-Lite radius/strength/limit	small radius, $\Delta \leq 0.12$

3.9 Color Space Conversion (CSC)

Overview

We convert display-encoded $R'G'B'$ (either uint8 or float) to digital $Y'CbCr$ 4:4:4 (planar, 8-bit). It supports BT.601/BT.709/BT.2020 coefficient sets and full/limited range mapping. Inputs are normalized to uint8 first: integer inputs use their native range; floating inputs are interpreted using `meta.white_level` (or 1.0 when the data already look like $[0, 1]$) to avoid unintended darkening. Optional artifacts include separate $Y/Cb/Cr$ PNGs, a triptych ($Y|Cb|Cr$), and a planar `.yuv` dump.

3.9.1 Notation

Let $x \in R^{H \times W \times 3}$ be the input and WL the effective white level. We use luma weights (K_r, K_g, K_b) and chroma rows $(u_r, u_g, u_b), (v_r, v_g, v_b)$ from the selected standard.

3.9.2 Step 1: Pre-normalization to 8-bit $R'G'B'$

$$x_{01} = \text{clip}\left(\frac{x}{\text{WL}}, 0, 1\right), \quad \mathbf{R'G'B'} = \text{round}(255 \cdot x_{01}) \in \{0, \dots, 255\}^3.$$

If the dtype is already uint8, this step is bypassed.

3.9.3 Step 2: RGB $\rightarrow$ Y'CbCr

With $\mathbf{R'G'B'} = (R', G', B')$,

$$Y' = K_r R' + K_g G' + K_b B', \quad Cb' = u_r R' + u_g G' + u_b B' + 128, \quad Cr' = v_r R' + v_g G' + v_b B' + 128,$$

then round/clip to $[0, 255]$.

3.9.4 Step 3: Range Mapping (full $\leftrightarrow$ limited)

For limited-range delivery,

$$Y_{\text{lim}} = 16 + \frac{219}{255}Y, \quad Cb_{\text{lim}} = 128 + \frac{224}{255}(Cb - 128), \quad Cr_{\text{lim}} = 128 + \frac{224}{255}(Cr - 128).$$

3.9.5 Step 4: Planar 4:4:4 Output

We store three $H \times W$ planes as Y then Cb then Cr .

3.9.6 Remarks

(i) Choose coefficients to match upstream primaries/OETF (BT.709 is common post tone/gamma). (ii) Keep full range internally and convert to limited only when required. (iii) Neutral chroma centers at 128; process in float, quantize last.

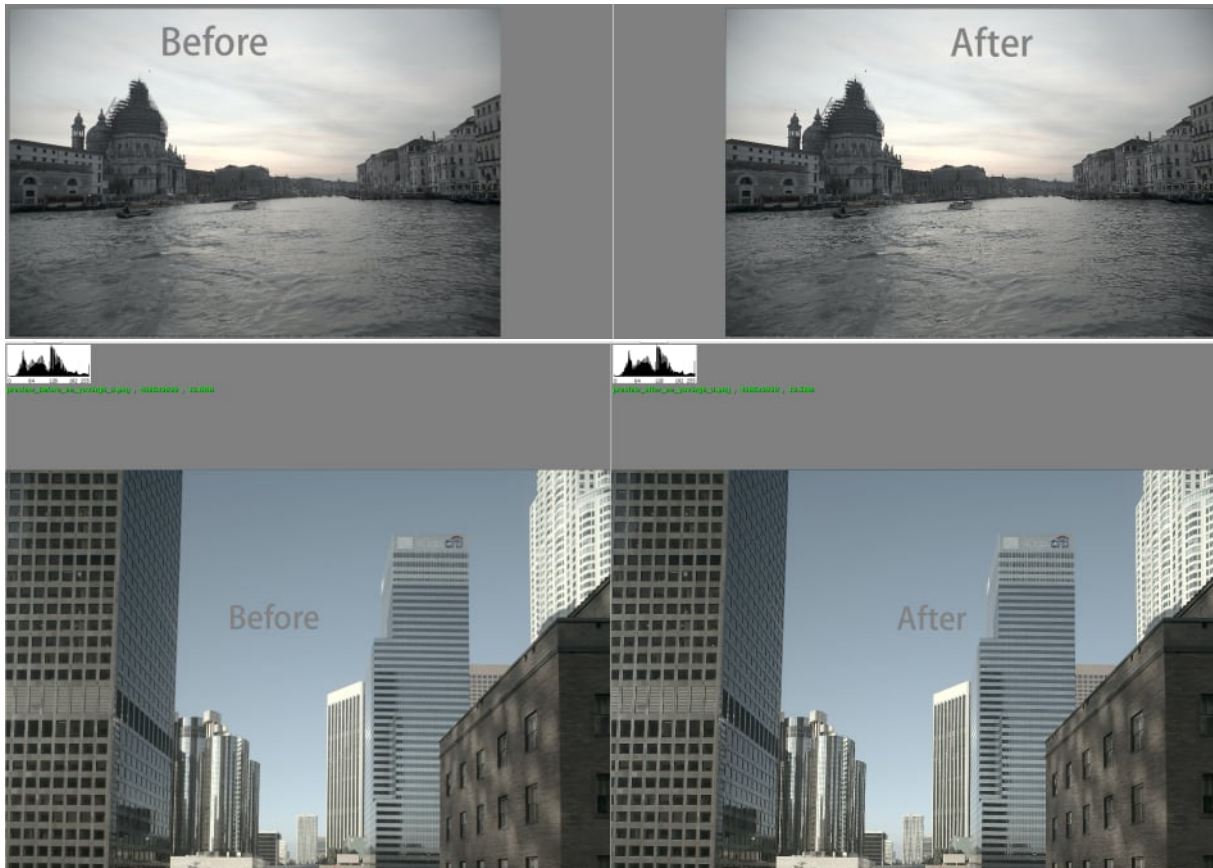
Table 3.3 CSC parameters and typical settings

Name	Meaning	Typical / Default
standard	Coefficient set	bt709 (display/video), bt601 (SD), bt2020 (HDR/WCG)
range_mode	Output range mapping	full (0..255) or limited (16..235/240)
white_level	Float input normalization WL	= 255 for 8-bit; = 1.0 when input in $[0, 1]$
clamp	Clip before quantization	on
save_raw/png/composite	Artifacts for QA	planar.yuv / per-plane PNG / triptych+stack

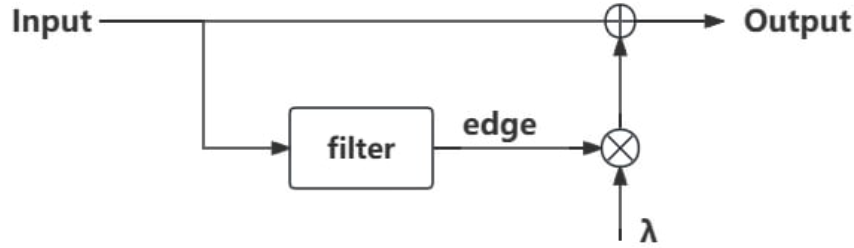
3.10 Sharpening (EE)

Overview

We enhance perceived sharpness by operating on the luminance channel Y' only, leaving chroma unchanged to avoid color artifacts. The method follows an unsharp-masking (USM) paradigm with *noise gating*, *edge masking*, *tone protection*, and *asymmetric halo limiting*.



3.10.1 Notation and Detail Layer



$$\text{Output}(x, y) = \text{Input}(x, y) + \lambda * \text{edge}$$

Let $Y_i \in [0, 1]$ be the display-referred luminance at pixel i . Build a base layer by Gaussian blur $B_i = (\mathcal{G}_\sigma * Y)_i$ and define the detail (high-pass) layer

$$D_i = Y_i - B_i. \quad (3.38)$$

Optionally soften the added detail later with a small blur $\mathcal{G}_{\sigma_s}$ to reduce ringing.

3.10.2 Noise-Gated Detail (MAD Threshold)

Estimate the global noise level from a high-pass response $H * Y$ via MAD:

$$\sigma_n = \frac{\text{MAD}((H * Y)_i)}{0.6745}, \quad \text{MAD}(z) = \text{median}(|z - \text{median}(z)|). \quad (3.39)$$

Apply a soft-threshold (shrinkage) to suppress sub-noise details:

$$\tilde{D}_i = \text{shrink}(D_i; \tau_n) = \text{sign}(D_i) \max(|D_i| - \tau_n, 0), \quad \tau_n = \lambda_n \sigma_n. \quad (3.40)$$

3.10.3 Edge Mask (Gradient-Weighted)

Compute gradients (e.g., Scharr/Sobel) and their magnitude $m_i = \sqrt{(G_x * Y)_i^2 + (G_y * Y)_i^2}$. Form a soft edge weight using robust percentiles $q_{p_1} = \text{perc}_{p_1}(m)$ and $q_{p_2} =$

$\text{perc}_{p_2}(m)$:

$$w_i^{(e)} = \text{clip}\left(\frac{m_i - q_{p_1}}{q_{p_2} - q_{p_1} + \epsilon}, 0, 1\right). \quad (3.41)$$

3.10.4 Tone Protection (Shadows/Highlights)

Reduce sharpening in deep shadows and near highlights to avoid noise boosting and highlight flattening. With smooth ramps

$$u(t; a, b) = \text{clip}\left(\frac{t - a}{b - a}, 0, 1\right), \quad (3.42)$$

define

$$w_i^{(t)} = u(Y_i; y_s^-, y_s^+) \left[1 - u(Y_i; y_h^-, y_h^+) \right]. \quad (3.43)$$

3.10.5 Asymmetric Halo Limiting

Form the preliminary increment

$$\Delta_i = k (\mathcal{G}_{\sigma_s} * \tilde{D})_i \cdot w_i^{(e)} \cdot w_i^{(t)}, \quad (3.44)$$

estimate a local contrast proxy c_i (e.g., $c_i = (\mathcal{G}_{r_c} * m)_i$), and clamp overshoot/undershoot asymmetrically:

$$u_+ = \alpha_+ c_i + \epsilon, \quad u_- = \alpha_- c_i + \epsilon, \quad \Delta_i^{\text{clp}} = \text{clip}(\Delta_i, -u_-, u_+). \quad (3.45)$$

3.10.6 Final Composition

Apply the enhancement on luminance and keep chroma intact:

$$Y'_{i,\text{out}} = \text{clip}(Y_i + \Delta_i^{\text{clp}}, 0, 1), \quad Cb_{\text{out}} = Cb, \quad Cr_{\text{out}} = Cr. \quad (3.46)$$

Table 3.4 Parameter quick reference —EE

Parameter	Purpose	Typical / Range
σ	Blur radius for base B	0.8–2.5 px
k	Global sharpening gain in (3.44)	0.2–1.2
λ_n	Noise gate factor in (3.40)	2–4
p_1, p_2	Edge percentiles in (3.41)	$p_1=60\text{--}75$, $p_2=90\text{--}97$
y_s^-, y_s^+	Shadow protection range	e.g. [0.05, 0.15]
y_h^-, y_h^+	Highlight protection range	e.g. [0.85, 0.97]
α_+, α_-	Asymmetric halo caps in (3.45)	$\alpha_+ = (4\sim 8) \times 10^{-2}$, $\alpha_- = (2\sim 6) \times 10^{-2}$
σ_s	Gentle pre-add smoothing of detail	0–1.0 px
r_c	Contrast pooling radius for c_i	small (e.g. 1–3 px)

3.11 Saturation

Overview

We adjust colorfulness by scaling the chroma vector in $Y'CbCr$ while preserving luminance. The operation respects full/limited code ranges and keeps neutral gray fixed.



3.11.1 Notation

Let $Y', Cb, Cr \in [0, 255]$ and the centered chroma

$$u = Cb - 128, \quad v = Cr - 128, \quad \mathbf{c} = (u, v).$$

Let s be the global saturation scale.

3.11.2 Step 1: Center and Scale

We scale chroma magnitude:

$$(u', v') = s(u, v).$$

3.11.3 Step 2: Re-encode and Clip

We form

$$Cb' = \text{clip}(128 + u', 0, 255), \quad Cr' = \text{clip}(128 + v', 0, 255), \quad Y'_{\text{out}} = Y',$$

or, for limited range, convert through the full-limited affine ramps before/after scaling to keep legal codes.

3.11.4 Remarks

(i) Operating in $Y'CbCr$ preserves luminance by construction. (ii) Moderate s in $[0.5, 1.5]$ maintains a natural palette. (iii) For highlight safety, saturation can be chained *after* tone mapping.

3.12 Contrast

Overview

We control scene rendition by shaping a luminance-like channel while leaving chroma unchanged. We provide three canonical forms—linear pivot, sigmoid S-curve, and power-law gamma—and an optional histogram strategy for difficult distributions.



Let $x \in [0, 1]$ be a normalized luminance proxy (Y' , V in HSV, or L^*). We produce $y \in [0, 1]$ with clipping at the end.

3.12.1 Step 1: Linear Around a Midpoint

$$y = (x - m)k + m, \quad m \approx 0.5, k > 0.$$

$k > 1$ increases contrast; $0 < k < 1$ reduces it.

3.12.2 Step 2: Sigmoid S-Curve

$$y = \frac{1}{1 + \exp(g(c - x))}, \quad c \approx 0.5, g > 0.$$

This steepens mid-tones while rolling off shadows/highlights.

3.12.3 Step 3: Power-Law (Gamma) Shaping

$$y = x^\gamma, \quad \gamma < 1 \text{ lifts mids, } \gamma > 1 \text{ compresses mids.}$$

3.12.4 Step 4: Histogram Strategies

For challenging inputs we may apply global HE or CLAHE on the luminance proxy, then gently blend back to avoid over-stretch.

3.12.5 Remarks

(i) A fixed midpoint keeps exposure stable across variants. (ii) Sigmoid and gamma are smooth and differentiable, which helps avoid banding. (iii) Histogram methods are powerful but should be rate-limited to prevent halos.

3.13 Hue

Overview

We adjust hue globally or within a selected hue window while preserving luminance. A compact RGB-space rotation implements global hue change; selective edits blend a rotated candidate using HSV-domain weights. A mild warm-highlight bias is optional.



3.13.1 Notation

Let $(Y, U, V) \in [0, 255]$ and $(R, G, B) \in [0, 1]$. We convert between YUV and RGB with the same standard used in CSC.

3.13.2 Step 1: YUV $\leftrightarrow$ RGB Consistency

We decode chroma around the neutral center and map to float RGB. All operations clip to $[0, 1]$ before returning to YUV to avoid out-of-gamut excursions.

3.13.3 Step 2: Global Hue Rotation

With rotation angle θ (deg) and saturation scale $\hat{s}$, we apply a fixed 3×3 hue-rotation matrix $H(\theta)$ followed by a luminance-preserving saturation matrix $S(\hat{s})$:

$$\text{rgb}_{\text{out}} = \text{clip}_{[0,1]}(\text{rgb}_{\text{in}} (S(\hat{s}) H(\theta))^T).$$

3.13.4 Step 3: Selective Rotation (Windowed Blend)

We compute HSV, define a hue window $[h_{\min}, h_{\max}]$ with a soft Gaussian weight in hue and smooth gates in value/saturation, generate a rotated candidate, and blend by $w \in [0, 1]$ based on these weights.

3.13.5 Step 4: Warm Highlights (Optional)

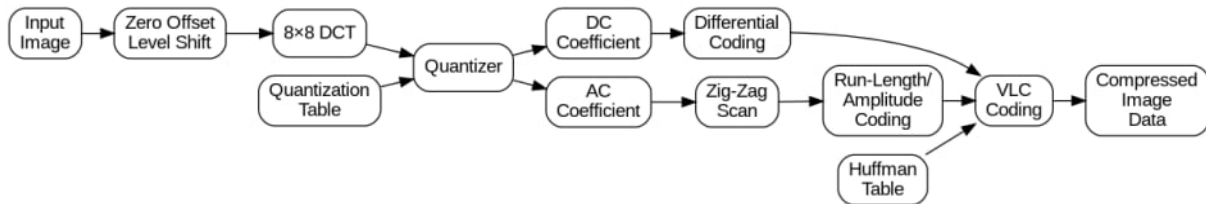
We gently nudge R up and B down as a function of value in bright regions and then re-encode to YUV.

3.13.6 Remarks

(i) Keeping $|\theta| \leq 15^\circ$ avoids unnatural skin-tone shifts. (ii) Selective windows should taper smoothly to prevent seams. (iii) All hue/sat operations honor the

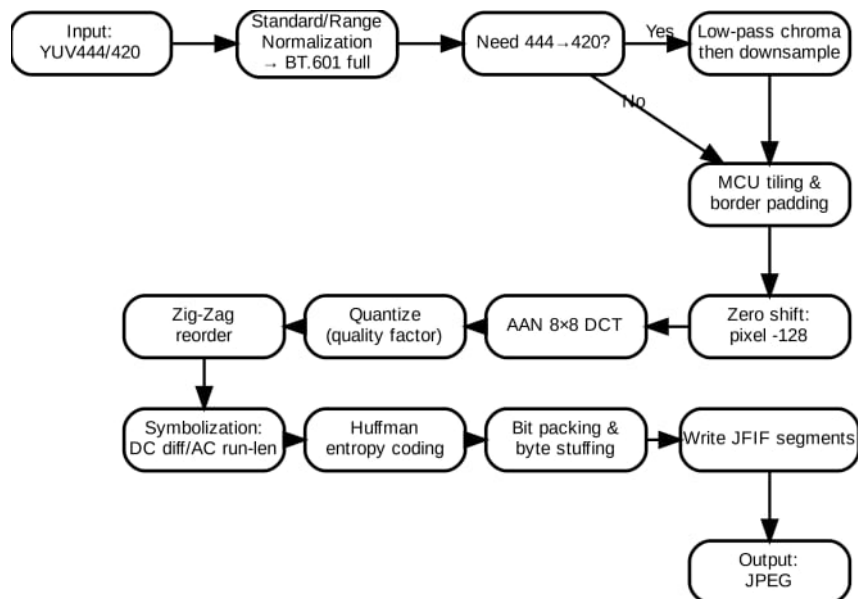
same BT.601/709/2020 choice as CSC.

3.14 YUV2JPEG



Overview

Encode 8-bit $YUV_{444/420}$ planes into a JFIF-compliant Baseline JPEG. The pipeline normalizes color standard/range, optionally downsamples chroma (444→420), tiles into MCUs, applies level shift and AAN FDCT, quantizes and reorders (ZigZag), RLE symbolization, Huffman entropy coding, byte stuffing, and writes SOI→APP0(JFIF)→DQT→SOF0→DHT→SOS→Entropy→EOI.





3.14.1 Color Standard & Range Normalization

Let input planes be (Y, U, V) with metadata (standard, range). Convert to a unified BT.601 full-range representation via an affine range expansion and a standard-aligned color transform:

$$Y^* = \alpha_Y Y + \beta_Y, \quad U^* = \alpha_U U + \beta_U, \quad V^* = \alpha_V V + \beta_V,$$

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \mathbf{T}_{\text{std} \rightarrow 601} \begin{bmatrix} Y^* \\ U^* \\ V^* \end{bmatrix}, \quad (3.47)$$

where $\mathbf{T}_{\text{std} \rightarrow 601}$ denotes the standard-to-BT.601 mapping (identity if already BT.601).

3.14.2 Optional 444→420 Downsampling

For chroma planes Cb, Cr , apply separable low-pass filtering with edge replication, then decimate by 2 along each axis:

$$C^{(h)} = \mathcal{F}_h * C, \quad C^\downarrow(m, n) = (\mathcal{F}_v * C^{(h)})(2m, 2n), \quad C \in \{Cb, Cr\}. \quad (3.48)$$

This matches common libjpeg-style quality/performance trade-offs.

3.14.3 MCU Layout and Padding

Let sampling factors be (s_x^Y, s_y^Y) for luma and (s_x^C, s_y^C) for chroma. The luma MCU size is $(8s_x^Y) \times (8s_y^Y)$; chroma MCUs follow their own grids. Pad each plane by edge replication to multiples of their MCU-aligned sizes:

$$W'_Y = \left\lceil \frac{W_Y}{8s_x^Y} \right\rceil 8s_x^Y, \quad H'_Y = \left\lceil \frac{H_Y}{8s_y^Y} \right\rceil 8s_y^Y, \quad W'_C = \left\lceil \frac{W_C}{8s_x^C} \right\rceil 8s_x^C, \quad H'_C = \left\lceil \frac{H_C}{8s_y^C} \right\rceil 8s_y^C. \quad (3.49)$$

3.14.4 Level Shift and 2D AAN FDCT

For each 8×8 block B , apply level shift $B'(x, y) = B(x, y) - 128$. The transform is the orthonormal 8×8 DCT (AAN implementation with separable scaling factors $S(u), S(v)$):

$$C(u, v) = \frac{1}{4} \alpha(u) \alpha(v) \sum_{x=0}^7 \sum_{y=0}^7 B'(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}, \quad \alpha(0) = \frac{1}{\sqrt{2}}, \alpha(k) = 1 \ (k > 0). \quad (3.50)$$

3.14.5 Quantization and Quality Control

Given luminance/chrominance quantization tables Q_Y, Q_C and a global quality scalar s_q , the effective tables are

$$\tilde{Q}_\chi(u, v) = \text{clip}(\text{round}(s_q Q_\chi(u, v) S(u) S(v)), 1, 255), \quad \chi \in \{Y, C\}. \quad (3.51)$$

Quantize by

$$\hat{C}_\chi(u, v) = \text{round}\left(\frac{C_\chi(u, v)}{\tilde{Q}_\chi(u, v)}\right). \quad (3.52)$$

For 4:4:4, optionally reuse luma table for chroma with a global scale to avoid overly soft chroma.

3.14.6 ZigZag Reorder, DC Differencing and AC RLE

Let $zz(k)$ be the ZigZag index ($k=0..63$). Vectorize each block:

$$\mathbf{z}_\chi[k] = \hat{C}_\chi(zz(k)), \quad k = 0, \dots, 63. \quad (3.53)$$

DC is differentially coded with previous block in scan order:

$$\Delta D_n = \mathbf{z}_\chi^{(n)}[0] - \mathbf{z}_\chi^{(n-1)}[0], \quad S_{\text{DC}}(n) = \begin{cases} 0, & \Delta D_n = 0, \\ \lfloor \log_2 |\Delta D_n| \rfloor + 1, & \text{otherwise.} \end{cases} \quad (3.54)$$

AC uses run-length encoding: for $k=1..63$, accumulate leading zeros to RUN , then emit a pair (RUN, S_{AC}) and amplitude bits; long zero runs use ZRL ($RUN=16$), and trailing all-zero tail uses EOB.

3.14.7 Huffman Entropy Coding

Map DC/AC symbols via default Annex K Huffman tables $H_{DC}^Y, H_{AC}^Y, H_{DC}^C, H_{AC}^C$:

$$\mathcal{B} += H_{DC}^X(S_{DC}(n)) \parallel \text{amp}(\Delta D_n, S_{DC}), \quad \mathcal{B} += H_{AC}^X(RUN, S_{AC}) \parallel \text{amp}(v, S_{AC}), \quad (3.55)$$

where $\mathcal{B}$ is the bitstream and $\text{amp}(\cdot)$ denotes JPEG amplitude bits for given size.

3.14.8 Bitstream Packing and Byte Stuffing

Flush bits MSB-first into bytes; whenever a byte equals $0xFF$, insert a $0x00$ (byte stuffing) to avoid fake markers. Pad the tail with 1 bits to complete the last byte.

3.14.9 JFIF Segments and Sampling Factors

Write markers in order:

$$SOI \rightarrow APP0(JFIF) \rightarrow DQT \rightarrow SOF0 \rightarrow DHT \rightarrow SOS \rightarrow \text{Entropy} \rightarrow EOI, \quad (3.56)$$

where SOF0 carries component sampling factors so the decoder reconstructs the MCU layout correctly for 4:4:4 or 4:2:0.

Table 3.5 Parameter quick reference —YUV2JPEG

Parameter	Purpose	Typical / Notes
subsample	444 / 420 selection	444 keeps chroma detail; 420 saves bits
s_q	Global quality scalar	maps to $\tilde{Q}_\chi$ in (3.51)
$\mathcal{F}_h, \mathcal{F}_v$	Chroma LPF for 444→420	separable, edge replication
$S(u), S(v)$	AAN scaling factors	absorbed in $\tilde{Q}_\chi$
Huffman	Annex K default tables	per (Y/C, DC/AC) in (3.55)
padding	Plane edge replication	meet MCU multiples, (3.49)

References

1. Color Correction Matrix (CCM) —Algorithm Design [in Chinese]. Zhihu Zhuanlan. Available: <https://zhuanlan.zhihu.com/p/34562544>. Accessed: 2025-10-09.
2. ISP Pipeline Overview —YUV Stage [in Chinese]. Zhihu Zhuanlan. Available: <https://zhuanlan.zhihu.com/p/653352999>. Accessed: 2025-10-09.
3. Automatic White Balance Algorithms: A Comparison [in Chinese]. Zhihu (Tardis/BD). Available: https://www.zhihu.com/tardis/bd/art/270558150?source_id=1001. Accessed: 2025-10-09.
4. Automatic White Balance (AWB) Technology [in Chinese]. 2048ai.net. Available: <https://2048ai.net/681dd4dea5baf817cf4a3687.html>. Accessed: 2025-10-09.
5. ISP Image Processing —YUV-Domain Pipeline [in Chinese]. CSDN Blog (author: qrx941017). Available: <https://blog.csdn.net/qrx941017/article/details/131637811>. Accessed: 2025-10-09.
6. Horizon J5 Image Tuning (4): AWB/CCM [in Chinese]. CSDN Blog (author: weixin_44690935). Available: https://blog.csdn.net/weixin_44690935/article/details/141951054. Accessed: 2025-10-09.
7. DPC (Dead Pixel Correction) —Introduction [in Chinese]. CSDN Blog (author: YUelite). Available: <https://blog.csdn.net/YUelite/article/details/139933471>. Accessed: 2025-10-09.
8. HiSilicon ISP Modules —Intro/Overview [in Chinese]. DeepinOut. Available: <https://deepinout.com/isp/hisi-isp-module-intro.html>. Accessed: 2025-10-09.
9. ISP —Gamma Correction [in Chinese]. CSDN Blog (author: wtzhu_13). Available: https://blog.csdn.net/wtzhu_13/article/details/119533870. Accessed: 2025-10-09.
10. ISP —Pipeline Overview [in Chinese]. CSDN Blog (author: wtzhu_13). Available: https://blog.csdn.net/wtzhu_13/article/details/118255864. Accessed: 2025-10-09.

11. Zhihu Zhuanlan article [in Chinese]. Available: <https://zhuanlan.zhihu.com/p/536544215>. Accessed: 2025-10-09.
12. Zhihu Zhuanlan article [in Chinese]. Available: <https://zhuanlan.zhihu.com/p/686901150>. Accessed: 2025-10-09.
13. Zhihu Zhuanlan article [in Chinese]. Available: <https://zhuanlan.zhihu.com/p/21983679>. Accessed: 2025-10-09.
14. FFmpeg Developers. (2023). *FFmpeg* [Computer software], Version 6.0. Retrieved from <https://ffmpeg.org/>. Accessed: 2025-10-09.